

Informed Search: Iterative Deepening A* and Depth First Branch & Bound Part-5

Introduction

In the realm of artificial intelligence, particularly in search problems, algorithms like A are celebrated for their ability to find optimal solutions efficiently by leveraging heuristic information. However, a significant challenge with A is its memory footprint; in the worst case, it can require exponential space to store the nodes it explores. This memory demand can quickly become prohibitive for complex problems, mirroring the issues encountered with algorithms like Breadth-First Search.

To address the memory limitations of A, researchers have developed alternative informed search strategies that aim to retain A's efficiency and optimality guarantees while operating within more constrained memory budgets. This lecture delves into two such memory-efficient algorithms: Iterative Deepening A (IDA) and Depth First Branch & Bound. These algorithms offer distinct approaches to balancing optimality, completeness, and memory usage, providing practitioners with powerful tools for tackling real-world search problems where resources are finite. Understanding their mechanics, advantages, and trade-offs is crucial for selecting the most appropriate search strategy for a given task.

Core Concepts

Term	Definition	Significance
Iterative Deepening A* (IDA*)	A memory-efficient variant of A* that performs a series of depth-first searches, each with an increasing F-value cutoff.	Addresses A*'s exponential memory problem by using depth-first search's linear memory, while maintaining optimality for admissible heuristics.
Depth First Branch & Bound	A depth-first search algorithm that uses both branching (heuristic node selection) and	Finds optimal solutions with linear memory by aggressively pruning branches that cannot lead to

Term	Definition	Significance
	bounding (pruning based on cost estimates) mechanisms.	a better solution than the current best found.
Branching Mechanism	A heuristic strategy within Depth First Branch & Bound that dictates the order in which child nodes are explored.	Guides the depth-first search towards promising paths, potentially finding a good solution faster to establish an initial upper bound.
Bounding Mechanism	A technique in Depth First Branch & Bound that prunes search branches if their estimated cost (current path cost + heuristic) exceeds a known upper bound.	Crucial for memory efficiency and speed, as it avoids exploring subtrees that cannot contain an optimal or better solution.
Global Upper Bound	The cost of the best solution found so far in Depth First Branch & Bound, initially set to infinity.	Acts as a dynamic threshold; any path whose estimated total cost exceeds this bound can be pruned, accelerating the search.
Lower Bound	The estimated minimum cost to reach a goal from a given node, typically calculated as $g(n) + h(n)$.	Used in conjunction with the global upper bound to determine if a branch can be pruned, ensuring optimality is not missed.
Weighted A*	A suboptimal variant of A* that modifies the evaluation function to $g(n) + w * h(n)$, where w is a weight factor.	Trades optimality for speed by giving more emphasis to the heuristic, often finding good solutions much faster with a guaranteed suboptimality bound.
Admissible Heuristic	A heuristic function $h(n)$ that never overestimates the true cost to reach the goal from node n .	Essential for guaranteeing optimality in A and IDA, as it ensures that pruning

Term	Definition	Significance
		decisions do not discard the optimal path.
Systematic Search	A search algorithm that guarantees every reachable state is visited exactly once (or at least not revisited unnecessarily) during its execution.	Important for efficiency; IDA* is not systematic due to restarts, while Depth First Branch & Bound is.
Optimal Solution	The solution with the lowest possible cost among all valid solutions to a search problem.	The primary goal of A, IDA, and Depth First Branch & Bound (when using admissible heuristics), though Weighted A* sacrifices it for speed.

Detailed Analysis

1. Iterative Deepening A (IDA)

The A algorithm, while optimal and complete with an admissible heuristic, suffers from high memory consumption, potentially requiring exponential space. To mitigate this, Iterative Deepening A (IDA) was developed as a memory-efficient alternative. The core intuition behind IDA is to mimic A*'s expansion order (nodes with lower F-values first) but within the memory constraints of a depth-first search.

- **Addressing Memory Issues:** A keeps all generated nodes in memory, leading to exponential space complexity in the worst case. IDA leverages the memory efficiency of depth-first search, which only needs to store the current path, resulting in linear space complexity.
- **F-value Cutoff:** Instead of a depth limit (as in standard Iterative Deepening Depth-First Search), IDA* uses an F-value cutoff. It performs a series of depth-first searches, where each search expands only nodes whose $f(n) = g(n) + h(n)$ value is less than or equal to the current cutoff.
- **Iteration Process:**
 - The first iteration's cutoff is typically the F-value of the start node.

- If a goal is not found within the current cutoff, the cutoff is increased to the minimum F-value of any node that was pruned in the previous iteration (i.e., nodes whose F-value exceeded the previous cutoff).
- The search then restarts from the beginning with this new, higher cutoff.
- **Optimality and Completeness:** Like A, IDA is optimal and complete if the heuristic is admissible. It guarantees that the first goal found will be an optimal one because it systematically explores nodes in increasing order of their F-values.
- **Systematicity:** IDA is not* systematic. Because it restarts each iteration with a new cutoff, it re-expands nodes multiple times. While this leads to redundant computation, the memory savings often outweigh the computational cost, especially in problems with large state spaces.
- **Performance:** IDA* is particularly well-suited for problems with infinite depth graphs, as its depth-first nature prevents it from getting lost in infinitely deep paths, and its F-value bound ensures progress towards the goal.

2. Depth First Branch & Bound

Depth First Branch & Bound (DFBnB) is another memory-efficient informed search algorithm that combines depth-first traversal with intelligent pruning. It is often used when finding an optimal solution is critical, but memory is a concern.

- **Two Core Mechanisms:**
- **Branching Mechanism:** This is a heuristic strategy that guides the depth-first search. It determines which child node to explore first from the current node. For example, one common strategy is to explore the child connected by the lowest-cost edge first. This mechanism aims to find any solution quickly to establish an initial upper bound.
- **Bounding Mechanism:** This is the key to DFBnB's efficiency. It involves maintaining two types of bounds:
- **Global Upper Bound:** This stores the cost of the best (lowest cost) goal path found so far. Initially, it's set to infinity. As soon as the first goal is reached, this bound is updated to the cost of that path. Subsequent goal paths found that are better will further reduce this upper bound.
- **Lower Bound for Current Path:** For any node n being expanded, its lower bound is calculated as $g(n) + h(n)$ (current path cost to n plus the heuristic estimate from n to the goal).
- **Pruning Logic:** If the lower bound of a node n ($g(n) + h(n)$) is greater than or equal to the current global upper bound, then that node and its

entire subtree can be pruned. This is because any path through n to the goal will necessarily be at least as expensive as $g(n) + h(n)$, and if this is already worse than or equal to the best solution found, there's no need to explore it further.

- **Example Walkthrough:** Consider a search from A to G, with an optimal path A-D-F-G costing 4.
- Start at A. Global upper bound = infinity.
- Using a branching mechanism (e.g., lowest cost edge first), explore A $\rightarrow$ B $\rightarrow$ H $\rightarrow$ G.
- A path A-B-H-G is found with cost 5. The global upper bound is updated to 5.
- Backtrack to B. Explore other children of B. Suppose B has a child E. If $g(E) + h(E)$ (e.g., $6 + 4 = 10$) is greater than the current upper bound (5), then the subtree rooted at E is pruned.
- Backtrack to A. Explore A $\rightarrow$ D. $g(D) + h(D)$ (e.g., $2 + 1 = 3$) is less than the upper bound (5), so continue.
- From D, explore D $\rightarrow$ F $\rightarrow$ G.
- A new path A-D-F-G is found with cost 4. The global upper bound is updated to 4.
- Backtrack to D. Explore other children of D (e.g., H). If $g(H) + h(H)$ (e.g., $9 + 1 = 10$) is greater than the new upper bound (4), prune H's subtree.
- Backtrack to A. Explore A $\rightarrow$ C. $g(C) + h(C)$ (e.g., $3 + 3 = 6$) is greater than the current upper bound (4). Prune the entire subtree rooted at C without visiting any of its children.
- The algorithm continues until all relevant paths have been explored or pruned, guaranteeing the optimal solution.
- **Systematicity:** DFBnB is systematic because it performs a single depth-first traversal, ensuring each node is visited at most once (unless it's part of a path that's later pruned).
- **Memory:** Like IDA*, DFBnB has linear space complexity because it's fundamentally a depth-first search.

3. Comparison of IDA* and Depth First Branch & Bound

Both IDA and DFBnB are designed to be memory-efficient alternatives to A, but they achieve this through different mechanisms and exhibit distinct performance characteristics.

- **Memory Properties:** Both algorithms offer linear space complexity, making them suitable for problems where A would exhaust memory. This is a significant advantage over A.
- **Optimality:** Both are optimal if they use an admissible heuristic and complete their search. They achieve optimality by ensuring that no optimal path is pruned.
- **Systematicity:**
 - IDA is not* systematic. It restarts the search from the beginning in each iteration, re-expanding nodes multiple times. This can lead to redundant computation but is effective in managing memory.
 - DFBnB is systematic. It performs a single depth-first traversal, and once a node is processed or pruned, it's not revisited in the same way.
- **Performance in Different Scenarios:**
 - **Infinite Depth Graphs:** IDA* tends to perform better in graphs with potentially infinite depth. Its F-value cutoff prevents it from getting stuck in deep, unpromising branches, whereas DFBnB, if its branching mechanism leads it down an infinite path before finding a good upper bound, could struggle.
 - **Ease of Finding Suboptimal Solutions:** DFBnB can be very efficient if it's easy to find a "good enough" (suboptimal) solution early in the search. Finding such a solution quickly establishes a tight global upper bound, leading to aggressive pruning and faster convergence to the optimal solution. If the first solution found is very poor, DFBnB might explore many nodes before pruning becomes effective.
 - **Initial Upper Bound Estimate:** If there's domain knowledge to provide a good initial estimate for the global upper bound, DFBnB can start pruning much earlier, significantly speeding up the search. Without such an estimate, it starts with an infinite bound, which is less effective initially.
 - **Many Paths to Goal:** If there are many paths to the goal, and many of them are close to optimal, DFBnB might perform well because it can quickly find a good upper bound. IDA* will still systematically explore F-layers.

4. Weighted A*

While IDA and DFBnB aim for optimality with memory efficiency, sometimes optimality is a luxury that can be traded for speed. Weighted A is an algorithm that explicitly makes this trade-off, sacrificing strict optimality for faster solution discovery.

- **Modified Evaluation Function:** Instead of $f(n) = g(n) + h(n)$, Weighted A* uses $f(n) = g(n) + w * h(n)$, where w is a weight factor.
- **Role of the Weight w :**
 - If $w = 1$, Weighted A is equivalent to standard A.
 - If $w > 1$, the heuristic component $h(n)$ is emphasized more. This makes the algorithm more "greedy," prioritizing nodes that appear closer to the goal according to the heuristic. This often leads to faster exploration but might miss the truly optimal path.
 - If $w < 1$, the path cost $g(n)$ is emphasized more. This makes the algorithm less greedy and potentially slower, but it might still not guarantee optimality.
- **Practical Application:** In practice, w is typically chosen to be greater than 1 (e.g., 5, 7, 10). This encourages the search to move more directly towards the goal, avoiding extensive exploration of side paths that A* might consider.
- **Optimality Bound:** A key property of Weighted A* is that if $w > 1$, the path found is guaranteed to be no worse than w times the optimal path cost. For example, if $w = 5$, the solution found will be at most 5 times the cost of the optimal solution. This provides a quantifiable bound on the suboptimality.
- **Benefits:** Weighted A finds solutions much faster than A in many practical scenarios, making it a popular choice when a "good enough" solution quickly is preferred over a strictly optimal one after extensive computation. It effectively balances the intuition of the heuristic with the accumulated path cost, but with a bias towards the heuristic.

Key Takeaways

- [] A* algorithm, while optimal, can suffer from exponential memory consumption in the worst case.
- [] Iterative Deepening A (IDA) addresses A*'s memory problem by performing a series of depth-first searches with increasing F-value cutoffs, achieving linear space complexity.

- [] IDA* is optimal and complete with an admissible heuristic but is not systematic due to restarting searches.
- [] Depth First Branch & Bound (DFBnB) uses a depth-first search combined with branching and bounding mechanisms to find optimal solutions with linear memory.
- [] DFBnB's bounding mechanism prunes subtrees whose estimated cost ($g(n) + h(n)$) exceeds the current best solution found (global upper bound).
- [] DFBnB is systematic and can be highly efficient if a good initial upper bound is known or a suboptimal solution is found early.
- [] Weighted A* is a suboptimal algorithm that prioritizes speed by modifying the evaluation function to $g(n) + w * h(n)$, where $w > 1$ emphasizes the heuristic.
- [] Weighted A* provides an optimality bound, guaranteeing the found solution is no worse than w times the optimal cost.
- [] The choice between IDA, DFBnB, and Weighted A depends on the specific problem characteristics, such as graph depth, ease of finding initial solutions, and the acceptable trade-off between optimality and speed.

Conclusion

The exploration of Iterative Deepening A, Depth First Branch & Bound, and Weighted A highlights the critical trade-offs inherent in designing effective search algorithms. While A provides a gold standard for optimality, its memory demands often necessitate more resource-conscious approaches. IDA and DFBnB offer elegant solutions to the memory problem, maintaining optimality by carefully managing the search space within linear memory constraints. Their differences in systematicity and performance under varying graph structures underscore the importance of understanding their underlying mechanics.

Furthermore, Weighted A* introduces the pragmatic notion that strict optimality is not always the primary goal. By allowing a controlled deviation from optimality, it can significantly accelerate the search process, yielding "good enough" solutions much faster. This flexibility is invaluable in real-world applications where computational time is a precious resource. The journey through these algorithms also sets the stage for the next crucial question: how do we effectively design and discover admissible heuristics? The ability to create strong, yet admissible, heuristics is fundamental to the power of all informed search techniques, and this will be the focus of future discussions.